# Programmazione a oggetti in Java

Appunti riassuntivi: classi e oggetti, costruttori, incapsulamento, metodi e `static`, input, classi wrapper, eccezioni, date, operatori, `toString()` e diagrammi UML

**Come leggere questi appunti.** I riquadri blu contengono le **definizioni** da sapere, quelli verdi gli **esempi**, quelli gialli i punti in cui è **facile sbagliare**. I blocchi grigi sono codice Java. In fondo trovi uno schema riassuntivo e dodici domande di autoverifica con le risposte.

## 1. Classi e oggetti

> **DEFINIZIONE**
> Una **classe** è il modello che descrive un tipo di oggetto: quali dati contiene (**attributi**) e cosa sa fare (**metodi**). Un **oggetto** è un esemplare concreto creato a partire dalla classe, detto anche **istanza**.

```
public class Libro {
    public String titolo;   // attributi
    public String autore;
    public int anno;
}
```

### 1.1 Creare un oggetto con new

```
Libro l1 = new Libro();
l1.titolo = "Il barone rampante";
l1.autore = "Italo Calvino";
l1.anno = 1957;
```

L'operatore `new` fa tre cose: riserva la memoria per il nuovo oggetto, esegue il **costruttore** e restituisce un **riferimento** all'oggetto, che viene salvato nella variabile. Agli attributi e ai metodi di un oggetto si accede con il punto: `l1.titolo`. Se una classe non dichiara nessun costruttore, Java ne fornisce automaticamente uno vuoto (**costruttore di default**).

### 1.2 Riferimenti

Una variabile di tipo classe non contiene l'oggetto, ma un riferimento che lo indica.

```
Libro l2 = l1;                // nessun nuovo oggetto
l2.anno = 2000;
System.out.println(l1.anno);  // stampa 2000
```

> **ATTENZIONE**
> Assegnare un oggetto a un'altra variabile copia il **riferimento**, non l'oggetto: `l1` e `l2` indicano lo stesso libro. Lo stesso vale quando si passa un oggetto a un costruttore o a un metodo: chi lo riceve lavora sull'oggetto originale, non su una copia.

### 1.3 Classi e file

- Una classe `public` deve stare in un file con lo stesso nome: `public class Libro` → `Libro.java`.

- Una classe senza modificatore (`class Main`) è visibile solo nel suo package.
- Le classi nella stessa cartella (stesso package) si usano a vicenda senza `import`.

## 2. Tipi di dato e variabili

In Java ogni variabile ha un tipo dichiarato, che non cambia. I tipi si dividono in due famiglie: **primitivi** e **riferimento**.

### 2.1 Tipi primitivi

Contengono direttamente il valore. Sono otto:

| Tipo | Contiene | Dimensione | Esempio |
|---|---|---|---|
| `byte`, `short` | interi piccoli | 8 e 16 bit | `byte b = 100;` |
| `int` | interi (circa ±2 miliardi) | 32 bit | `int anno = 1957;` |
| `long` | interi molto grandi | 64 bit | `long ore = 3L;` |
| `float` | numeri con la virgola, precisione singola | 32 bit | `float f = 2.5f;` |
| `double` | numeri con la virgola, precisione doppia (il tipo decimale di default) | 64 bit | `double prezzo = 2.5;` |
| `char` | un carattere | 16 bit | `char c = 'A';` |
| `boolean` | `true` o `false` | — | `boolean ok = true;` |

### 2.2 Tipi riferimento e null

Tutto ciò che non è primitivo è un oggetto: `String`, le classi della libreria (`LocalDateTime`…) e le classi scritte da noi (`Libro`). La variabile contiene un riferimento all'oggetto. Il valore speciale `null` significa "nessun oggetto".

> **ATTENZIONE**
> Usare un attributo o un metodo attraverso un riferimento `null` lancia `NullPointerException`. Per controllare se un riferimento è vuoto si scrive `if (x == null)`.

### 2.3 Variabili locali e attributi

| | Variabile locale | Attributo |
|---|---|---|
| **Dove si dichiara** | dentro un metodo | nella classe, fuori dai metodi |
| **Quanto dura** | finché il metodo è in esecuzione | quanto l'oggetto |
| **Valore iniziale** | nessuno: va assegnato prima dell'uso | di default: `0`, `false` o `null` |

ATTENZIONE

Il compilatore rifiuta una variabile locale che **potrebbe** non essere stata assegnata:

```
int x;
if (scelta > 0) {
    x = 5;
}
System.out.println(x);   // errore: variable x might not have been initialized
```

Per questo, prima di un `if` o di un `try` in cui ricevono il valore vero, le variabili locali si inizializzano spesso con un valore neutro ( `""` , `0` , `0.0` ).

Gli array si possono dichiarare in due modi equivalenti: `String[] args` (stile Java) e `String args[]` (stile C).

## 3. Costruttori e this

DEFINIZIONE

Il **costruttore** è un metodo speciale che inizializza l'oggetto appena creato. Ha lo **stesso nome della classe**, **nessun tipo di ritorno** (neanche `void` ) e viene eseguito automaticamente da `new` .

```
public class Libro {
    public String titolo;
    public String autore;
    public int anno;

    public Libro(String titolo, String autore, int anno) {
        this.titolo = titolo;
        this.autore = autore;
        this.anno = anno;
    }
}

Libro l = new Libro("Il barone rampante", "Italo Calvino", 1957);
```

Parametro e attributo hanno lo stesso nome: dentro il costruttore `titolo` indica il **parametro**, che "copre" l'attributo. La parola chiave `this` indica l'**oggetto corrente**, quindi `this.titolo` è l'attributo. La riga `this.titolo = titolo;` si legge: "copia il parametro nell'attributo di questo oggetto".

Non tutti gli attributi devono essere impostati nel costruttore: quelli non assegnati mantengono il valore di default. È il caso di un dato che si conosce solo più tardi (per esempio la data di restituzione di un prestito), che resta `null` finché un metodo non lo imposta.

ATTENZIONE

- Gli argomenti vengono associati ai parametri **per posizione**, non per nome: `new Libro("Italo Calvino", "Il barone rampante", 1957)` compila (sono due `String` ), ma titolo e autore finiscono scambiati.
- Dopo aver scritto un costruttore con parametri, il costruttore di default non esiste più: `new Libro()` dà errore di compilazione.

# 4. Modificatori di accesso e incapsulamento

## 4.1 I modificatori

| Modificatore | Visibile da | Simbolo UML |
|---|---|---|
| `public` | qualsiasi classe | `+` |
| `protected` | stesso package e sottoclassi | `#` |
| *(nessuno)* | solo le classi dello stesso package | `~` |
| `private` | solo l'interno della stessa classe | `-` |

## 4.2 Incapsulamento

> **DEFINIZIONE**
> L'**incapsulamento** consiste nel nascondere gli attributi ( `private` ) e permettere di usarli solo attraverso metodi pubblici scelti dalla classe. Così è la classe a decidere che cosa si può fare sui suoi dati.

```
public class ContoCorrente {
    private String intestatario;
    private double saldo;

    public ContoCorrente(String intestatario) {
        this.intestatario = intestatario;
    }

    public double getSaldo() {                        // getter
        return saldo;
    }

    public void setIntestatario(String intestatario) {  // setter
        this.intestatario = intestatario;
    }

    public void deposita(double importo) {
        if (importo > 0) {
            saldo += importo;
        }
    }
}
```

> **ESEMPIO**
> Da un'altra classe `conto.saldo = 1000;` non compila (*saldo has private access in ContoCorrente*). Il saldo si può leggere con `getSaldo()` ma non impostare direttamente, perché non esiste `setSaldo` : l'unico modo per aumentarlo è `deposita()` , che ignora gli importi negativi. Con attributi `public` , come in `Libro` , chiunque potrebbe invece scrivere qualsiasi valore.

### 4.3 Getter e setter

| | Getter | Setter |
|---|---|---|
| **Scopo** | leggere un attributo privato | impostare un attributo privato |
| **Nome** | `getNome()` (`isNome()` per i boolean) | `setNome(valore)` |
| **Ritorno** | il tipo dell'attributo | `void` |

## 5. Metodi

Un metodo è un blocco di codice con un nome che si può richiamare. La sua **firma** dice tutto ciò che serve per usarlo:

| `public` | `double` | `getSaldo` | `( )` |
|---|---|---|---|
| modificatore di accesso | tipo di ritorno | nome | parametri (qui nessuno) |

- `void`: il metodo non restituisce nulla.
- Se il tipo di ritorno non è `void`, il metodo deve terminare con `return` seguito da un valore di quel tipo.
- I **parametri** sono le variabili della dichiarazione (`double importo`); gli **argomenti** sono i valori passati nella chiamata (`conto.deposita(50)`).

### 5.1 Metodi d'istanza e metodi statici

| | Metodo d'istanza | Metodo statico |
|---|---|---|
| **Si chiama su** | un oggetto: `conto.deposita(50)` | la classe: `Double.parseDouble("2.5")` |
| **Usa gli attributi dell'oggetto** | sì | no (non c'è un oggetto) |
| **Esempi** | `getSaldo()`, `toString()` | `main`, `LocalDateTime.now()` |

## 6. static e il metodo main

> DEFINIZIONE
> Un membro `static` appartiene alla **classe** e non ai singoli oggetti: esiste in una sola copia e si può usare senza creare oggetti. Nei diagrammi UML è **sottolineato**.

```
public static void main(String[] args) {
    // qui inizia l'esecuzione del programma
}
```

| Parola | Perché c'è |
|---|---|
| `public` | la JVM (Java Virtual Machine), che sta fuori dalla classe, deve poterlo chiamare |
| `static` | la JVM lo esegue senza creare un oggetto della classe |
| `void` | non restituisce nulla |
| `String[] args` | gli eventuali argomenti passati da riga di comando |

**ESEMPIO**

`System.out.println("Ciao")`: `System` è una classe, `out` è un suo attributo **statico** (di tipo `PrintStream`) e `println` è un metodo d'istanza chiamato sull'oggetto `out`.

## 7. Package e import

**DEFINIZIONE**

Un **package** è un gruppo di classi correlate, organizzato come una cartella. Il nome completo di una classe comprende il suo package: `java.time.LocalDateTime`. Con `import` la si può poi usare con il nome corto.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;
```

| Package | Contiene | Esempi |
|---|---|---|
| `java.lang` | le classi di base; importato **automaticamente** | `String`, `System`, `Double`, `Integer` |
| `java.io` | input e output | `BufferedReader`, `InputStreamReader`, `IOException` |
| `java.time` | date e orari | `LocalDate`, `LocalDateTime` |
| `java.time.temporal` | unità e calcoli sul tempo | `ChronoUnit` |

**ATTENZIONE**

`import java.time.*;` importa tutte le classi di `java.time`, ma **non** quelle dei sotto-package: `java.time.temporal.ChronoUnit` va importata a parte.

## 8. Input da tastiera: gli stream

**DEFINIZIONE**

Uno **stream** (flusso) è una sequenza di dati che scorre da una sorgente (tastiera, file, rete) verso il programma, o dal programma verso una destinazione (schermo, file).

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

In una sola riga vengono creati due oggetti, uno "avvolto" nell'altro. Ogni livello aggiunge una funzione a quello che contiene:

`System.in`
flusso di **byte** grezzi che arrivano dalla tastiera

→

`InputStreamReader`
converte i byte in **caratteri**

→

`BufferedReader`
aggiunge un **buffer** e la lettura comoda del testo

Questo modo di comporre gli oggetti, in cui un oggetto ne avvolge un altro per aggiungergli capacità, è tipico di `java.io` e si chiama pattern **decorator**. Il `BufferedReader` si crea una volta e si riusa per tutte le letture. Due cose da ricordare: quello che arriva dalla tastiera è sempre **testo** (`String`, vedi cap. 9) e le operazioni di lettura possono lanciare `IOException` (vedi cap. 10).

## 9. Classi wrapper e conversioni

> **DEFINIZIONE**
> Ogni tipo primitivo ha una **classe wrapper** corrispondente, che rappresenta lo stesso valore come oggetto e offre metodi statici di utilità, tra cui quelli per convertire una `String` in numero.

| Primitivo | Wrapper | Da String al primitivo |
|---|---|---|
| `int` | `Integer` | `Integer.parseInt("1957")` |
| `long` | `Long` | `Long.parseLong("10")` |
| `double` | `Double` | `Double.parseDouble("2.5")` |
| `boolean` | `Boolean` | `Boolean.parseBoolean("true")` |
| `char` | `Character` | — |

```
String testo = "2.50";
double prezzo = Double.parseDouble(testo);   // 2.5
int anno = Integer.parseInt("1957");          // 1957
```

> **ATTENZIONE**
> Se il testo non è un numero valido, i metodi `parse…` lanciano `NumberFormatException`. Il separatore decimale è sempre il **punto**: `"2.50"` funziona, `"2,50"` e `"abc"` no.

Per il percorso inverso, da numero a testo, basta la concatenazione (`"" + 2.5`) oppure `String.valueOf(2.5)`.

## 10. Eccezioni

> **DEFINIZIONE**
> Un'**eccezione** è un oggetto che rappresenta un errore avvenuto durante l'esecuzione. Quando viene **lanciata**, il flusso normale si interrompe: se qualcuno la **cattura** il programma prosegue da lì, altrimenti termina stampando il messaggio d'errore.

### 10.1 Checked e unchecked

Le eccezioni sono classi che derivano da `Exception` (a sua volta figlia di `Throwable`). Quelle che derivano da `RuntimeException` sono **unchecked**, tutte le altre sono **checked**.

| | Checked | Unchecked |
|---|---|---|
| **Gestione** | **obbligatoria** (`try/catch` o `throws`), altrimenti il codice non compila | facoltativa |
| **Indicano di solito** | problemi esterni al programma (input, file, rete) | errori di logica o dati non validi |
| **Esempi** | `IOException` | `NumberFormatException`, `NullPointerException`, `IllegalStateException` |

### 10.2 try / catch

```
try {
    double prezzo = Double.parseDouble(testo);
    System.out.println("Prezzo: " + prezzo);
} catch (NumberFormatException e) {
    System.out.println("Non è un numero: " + e.getMessage());
}
System.out.println("Il programma continua");
```

Si eseguono le istruzioni del `try`. Se una lancia un'eccezione, le successive del `try` vengono **saltate** e si esegue il `catch` corrispondente; dopo il `catch` il programma continua normalmente. Dentro il `catch`, `e.getMessage()` restituisce il messaggio dell'errore e `e.printStackTrace()` stampa il tipo di eccezione e l'elenco dei metodi in cui è avvenuta.

> **ATTENZIONE**
> Un `catch` cattura **solo** il tipo indicato (e le sue sottoclassi). Un `catch (IOException e)` non ferma una `NumberFormatException` lanciata nello stesso `try`: il programma termina comunque.

### 10.3 Le eccezioni checked: gestire o dichiarare

```
// 1) gestirla sul posto
try {
    // operazioni di input che possono lanciare IOException
} catch (IOException e) {
    e.printStackTrace();
}

// 2) dichiararla nella firma: la dovrà gestire chi chiama il metodo
public static void main(String[] args) throws IOException {
    // ...
}
```

### 10.4 Lanciare un'eccezione: throw

```
public double calcolaMedia() {
    if (numeroVoti == 0) {
        throw new IllegalStateException("Nessun voto inserito");
    }
    return sommaVoti / numeroVoti;   // sommaVoti è un double
}
```

Con `throw` è il programmatore a lanciare un'eccezione, creandola con `new` e passandole un messaggio. `IllegalStateException` indica che l'oggetto non è nello stato giusto per eseguire l'operazione richiesta: qui non si può calcolare una media senza voti.

> **ATTENZIONE**
> `throw` (senza s) è un'**istruzione** che lancia un'eccezione; `throws` (con la s) sta nella **firma** del metodo e dichiara quali eccezioni checked può lanciare.

## 11. Date e ore con java.time

| Classe | Rappresenta | Esempio di stampa |
|---|---|---|
| `LocalDate` | una data | `2026-10-12` |
| `LocalTime` | un orario | `10:30` |
| `LocalDateTime` | data e orario, senza fuso orario | `2026-10-12T10:30` |
| `ChronoUnit` | le unità di tempo ( `DAYS` , `HOURS` , `MINUTES` …) | — |

| Codice | Cosa fa |
|---|---|
| `LocalDateTime.now()` | metodo statico: data e ora attuali del computer |
| `LocalDateTime.of(2026, 10, 12, 10, 0)` | crea una data/ora precisa (anno, mese, giorno, ore, minuti) |
| `x.toLocalDate()` | estrae solo la data, scartando l'orario |
| `ChronoUnit.HOURS.between(a, b)` | quante ore **intere** ci sono tra `a` e `b` (restituisce un `long` ) |
| `a.equals(b)` | `true` se le due date sono uguali |

```
LocalDateTime inizio = LocalDateTime.of(2026, 10, 12, 10, 0);
LocalDateTime fine   = LocalDateTime.of(2026, 10, 12, 11, 59);

long ore    = ChronoUnit.HOURS.between(inizio, fine);     // 1
long minuti = ChronoUnit.MINUTES.between(inizio, fine);   // 119
boolean stessoGiorno = inizio.toLocalDate().equals(fine.toLocalDate());   // true
```

> **DEFINIZIONE**
> Gli oggetti di `java.time` sono **immutabili**: una volta creati non cambiano più. I metodi che sembrano modificarli, come `plusHours(2)` , restituiscono un **nuovo** oggetto.

> **ATTENZIONE**
> `between` conta solo le unità **complete** e tronca il resto: tra 10:00 e 11:59 c'è **1** ora, non 2. Quando stampi un `LocalDateTime` compare il formato ISO-8601, con una `T` tra data e ora.

# 12. Operatori ed espressioni

## 12.1 Gli operatori principali

| Tipo | Operatori | Esempio |
|---|---|---|
| **Assegnamento** | `=` | `ore = 1;` |
| **Aritmetici** | `+` `-` `*` `/` `%` | `ore * tariffa` |
| **Composti** | `+=` `-=` `*=` `/=` | `x *= 0.8;` equivale a `x = x * 0.8;` |
| **Confronto** | `==` `!=` `<` `>` `<=` `>=` | `if (ore == 0)` |
| **Logici** | `!` (NOT) `&&` (AND) `\|\|` (OR) | `if (!stessoGiorno)` |

## 12.2 Conversioni automatiche tra tipi numerici

In un'espressione con tipi numerici diversi, il tipo più "piccolo" viene convertito automaticamente in quello più "grande" (**promozione**): `int` → `long` → `float` → `double`.

> **ESEMPIO**
> `long ore = 3;` e `double tariffa = 1.5;` → `ore * tariffa` è un `double` e vale `4.5`. Il contrario non è automatico: per mettere un `double` in un `int` serve un cast, `(int) 4.5`, che tronca i decimali e dà `4`.

## 12.3 Concatenazione di stringhe

Se almeno un operando di `+` è una `String`, il `+` diventa **concatenazione** e l'altro operando viene convertito in testo. L'espressione si valuta da sinistra a destra:

```
System.out.println("a" + 1 + 2);   // a12
System.out.println(1 + 2 + "a");   // 3a
```

## 12.4 I numeri decimali non sono esatti

> **ATTENZIONE**
> I `double` sono memorizzati in binario e molti numeri decimali, come 0.1, non si possono rappresentare esattamente. `System.out.println(0.1 + 0.2)` stampa `0.30000000000000004`. Per i calcoli con il denaro nei programmi reali si usa la classe `BigDecimal`, oppure si arrotonda l'output.

# 13. Confrontare oggetti: == ed equals

> **DEFINIZIONE**
> Con gli oggetti, `==` confronta i **riferimenti** (sono lo stesso oggetto in memoria?), mentre `equals()` confronta il **contenuto**. Con i tipi primitivi si usa sempre `==`.

```
String a = new String("ciao");
System.out.println(a == "ciao");        // false: oggetti diversi
System.out.println(a.equals("ciao"));   // true: stesso contenuto
```

Lo stesso vale per le date: due `LocalDate` del 12 ottobre creati separatamente sono `equals` ma non `==`. Per controllare se un riferimento è `null` si usa invece `==`.

## 14. Object, toString() e @Override

> **DEFINIZIONE**
> Ogni classe Java **eredita** automaticamente dalla classe `Object`, che fornisce alcuni metodi già pronti tra cui `toString()`. Ridefinire in una classe un metodo ereditato si chiama **override**.

Il `toString()` di `Object` restituisce il nome della classe, una `@` e un codice esadecimale, per esempio `Libro@78308db1`. Ridefinendolo si ottiene un testo leggibile:

```
@Override
public String toString() {
    return titolo + " (" + autore + ", " + anno + ")";
}

System.out.println(l);   // Il barone rampante (Italo Calvino, 1957)
```

- `System.out.println(oggetto)` chiama **automaticamente** `oggetto.toString()`. Lo stesso succede concatenando un oggetto a una stringa: `"Libro: " + l`.
- `@Override` è un'**annotazione**: dice al compilatore che il metodo ne ridefinisce uno ereditato. Se si sbaglia nome o parametri (es. `tostring()`) il compilatore segnala l'errore, invece di creare in silenzio un metodo nuovo che non verrebbe mai chiamato da `println`.
- Un `toString()` può usare altri metodi della classe, ma se uno di questi lancia un'eccezione la lancia anche la stampa dell'oggetto.

### 14.1 Sequenze di escape

| Sequenza | Significato |
|---|---|
| `\n` | a capo |
| `\t` | tabulazione |
| `\"` | virgolette dentro una stringa |
| `\\` | il carattere `\` |

## 15. Relazioni tra classi e diagrammi UML

### 15.1 Associazione

> **DEFINIZIONE**
> C'è un'**associazione** tra due classi quando una ha un attributo del tipo dell'altra: è una relazione "**ha un**" (*has-a*). Un prestito *ha un* libro.

```java
public class Prestito {
    private Libro libro;            // associazione con Libro
    private LocalDate dataInizio;
    private LocalDate dataFine;     // null finché il libro non viene restituito

    public Prestito(Libro libro, LocalDate dataInizio) {
        this.libro = libro;
        this.dataInizio = dataInizio;
    }

    public void setDataFine(LocalDate dataFine) {
        this.dataFine = dataFine;
    }
}
```

La **navigabilità** dice chi conosce chi: `Prestito` conosce il suo `Libro`, ma `Libro` non ha attributi di tipo `Prestito` e non sa nulla dei prestiti. La **molteplicità** dice quanti oggetti sono collegati:

| Molteplicità | Significato |
|---|---|
| `1` | esattamente uno |
| `0..1` | nessuno o uno |
| `*` **oppure** `0..*` | un numero qualsiasi |
| `1..*` | almeno uno |

### 15.2 Dipendenza

Una classe **dipende** da un'altra quando la usa senza tenerla come attributo: per esempio un `main` che crea oggetti `Libro` e `Prestito` come variabili locali. È un legame più debole dell'associazione e in UML si disegna con una freccia **tratteggiata**; la scritta «create» (uno **stereotipo**) precisa che la classe crea gli oggetti dell'altra.

### 15.3 Leggere un diagramma delle classi

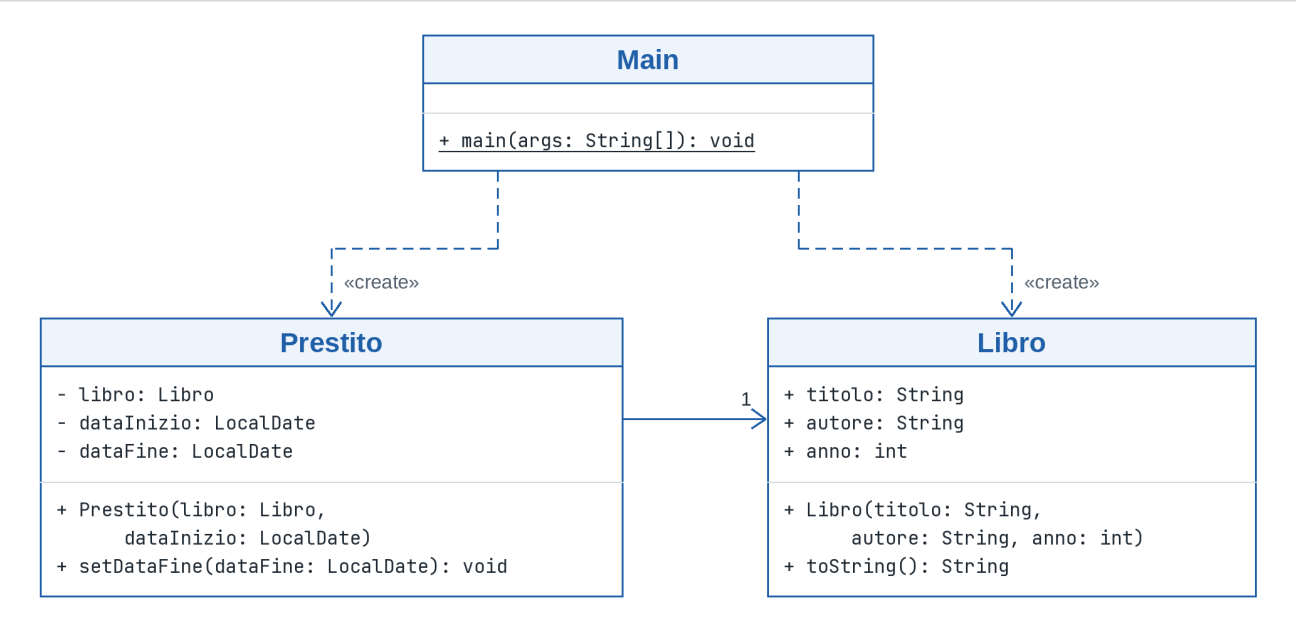



Figura 1 — Diagramma delle classi dell'esempio: Main, Prestito e Libro

| Elemento | Significato |
|---|---|
| **Riquadro a tre parti** | nome della classe · attributi · metodi |
| **`+` `-` `#` `~`** | `public`, `private`, `protected`, nessun modificatore |
| **`nome: Tipo`** | in UML il tipo si scrive **dopo** il nome, al contrario di Java (`String titolo`); per i metodi, dopo i `:` finali c'è il tipo di ritorno |
| **Testo sottolineato** | membro `static` |
| **Freccia piena con numero** | associazione navigabile nel verso della freccia, con la molteplicità |
| **Freccia tratteggiata «create»** | dipendenza: la classe crea oggetti dell'altra |

## 16. Schema riassuntivo

| Concetto | In breve |
|---|---|
| **Classe / oggetto** | modello / istanza concreta creata con `new` |
| **Riferimento** | la variabile indica l'oggetto; assegnarla non crea copie |
| **Primitivi / riferimenti** | valore diretto (`int`, `double`…) / riferimento a un oggetto (`String`…), che può essere `null` |
| **Locali / attributi** | le locali vanno inizializzate; gli attributi hanno valori di default |
| **Costruttore** | stesso nome della classe, nessun tipo di ritorno, eseguito da `new` |
| **`this`** | l'oggetto corrente; distingue l'attributo dal parametro omonimo |
| **Incapsulamento** | attributi `private` + metodi `public` (getter, setter, operazioni controllate) |
| **Firma di un metodo** | modificatore, tipo di ritorno, nome, parametri |
| **`static`** | appartiene alla classe; si usa senza oggetti (`main`, `parseDouble`, `now`) |
| **Package / import** | gruppi di classi; `java.lang` è importato da solo |
| **Stream** | oggetti avvolti: byte → caratteri → buffer (decorator) |
| **Wrapper** | classe di un primitivo; `parse…` converte da `String` |
| **Checked / unchecked** | gestione obbligatoria / facoltativa (`RuntimeException`) |
| **`try/catch`** | cattura un'eccezione e prosegue; solo il tipo indicato |
| **`throw` / `throws`** | lancia un'eccezione / la dichiara nella firma |
| **`java.time`** | date immutabili; `between` conta solo unità complete |
| **Promozione** | nei calcoli misti il tipo più piccolo diventa il più grande |
| **`==` / `equals`** | stesso oggetto / stesso contenuto |
| **Override** | ridefinire un metodo ereditato; `@Override` lo fa controllare al compilatore |
| **Associazione / dipendenza** | "ha un" (attributo) / usa o crea senza attributo |

## 17. Domande di autoverifica

**1. Che differenza c'è tra una classe e un oggetto?**
La classe è il modello che descrive attributi e metodi; l'oggetto è un'istanza concreta creata con `new`.

**2. Dopo `Libro b = a;` quanti oggetti `Libro` esistono?**
Uno solo: `a` e `b` contengono lo stesso riferimento, quindi modificare `b.anno` cambia anche `a.anno`.

**3. Perché `int x; System.out.println(x);` non compila?**
Le variabili locali non hanno un valore di default e vanno assegnate prima di essere usate.

**4. Che cosa fa `this.titolo = titolo;` in un costruttore?**
Copia il parametro `titolo` nell'attributo `titolo` dell'oggetto corrente; `this` distingue l'attributo dal parametro con lo stesso nome.

**5. Che cos'è l'incapsulamento?**
Rendere gli attributi `private` e permetterne l'uso solo tramite metodi pubblici, così la classe controlla come cambiano i suoi dati.

**6. Perché il metodo `main` è `static`?**
Perché la JVM deve poterlo eseguire senza creare un oggetto della classe.

**7. Che cosa succede con `Double.parseDouble("3,5")`?**
Viene lanciata `NumberFormatException`: il separatore decimale deve essere il punto.

**8. Che differenza c'è tra eccezioni checked e unchecked?**
Le checked vanno gestite con `try/catch` o dichiarate con `throws`, altrimenti il codice non compila; le unchecked (sottoclassi di `RuntimeException`) no.

**9. Che differenza c'è tra `throw` e `throws`?**
`throw` è l'istruzione che lancia un'eccezione; `throws` sta nella firma del metodo e dichiara le eccezioni checked che può lanciare.

**10. Quanto restituisce `ChronoUnit.HOURS.between` tra le 8:00 e le 9:45 dello stesso giorno?**
`1`: conta solo le ore complete.

**11. Perché due `String` si confrontano con `equals` e non con `==`?**
Perché `==` confronta i riferimenti (stesso oggetto), mentre `equals` confronta il contenuto.

**12. Che cosa stampa `System.out.println(libro)` se `Libro` non ridefinisce `toString()`?**
Il `toString()` ereditato da `Object`: nome della classe, `@` e un codice esadecimale, per esempio `Libro@78308db1`.